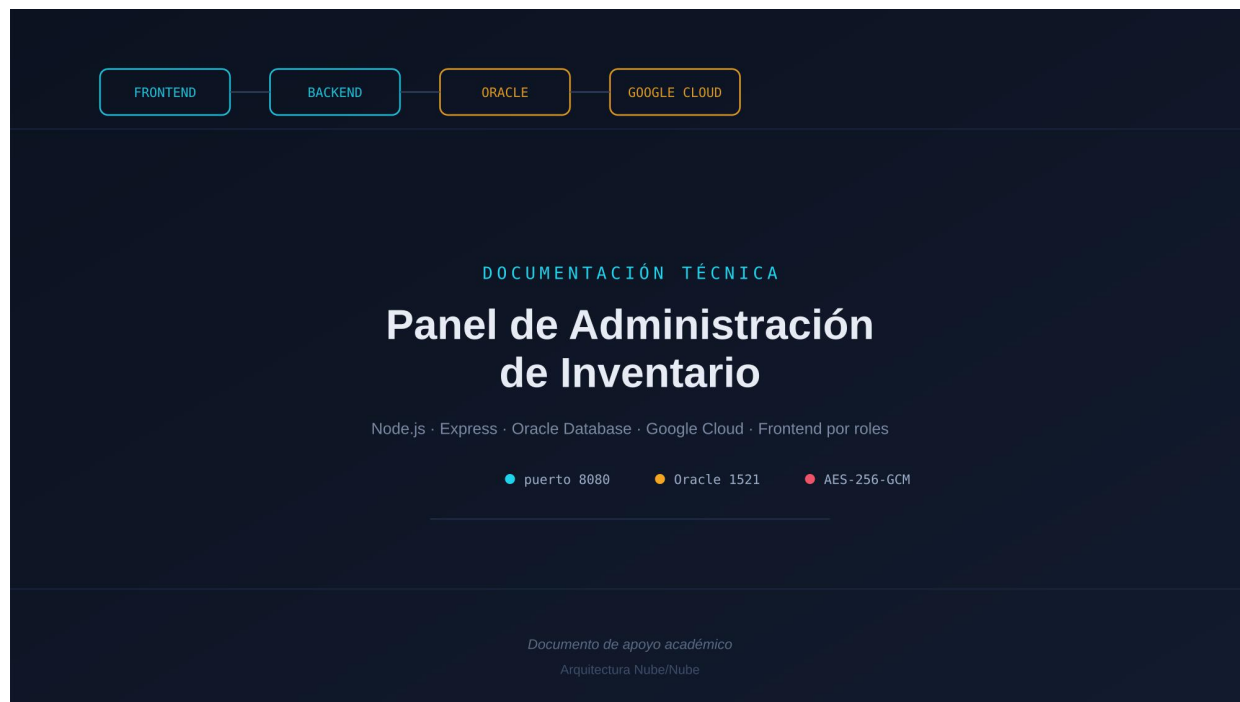




Documento de apoyo académico

1. Introducción

El presente documento desarrolla, con fines de apoyo académico, el diseño y la implementación de un sistema de control de inventario denominado Panel de Administración, construido sobre Oracle Database como motor de base de datos para el almacenamiento estructurado de usuarios, productos en stock y movimientos de entrada/salida. El sistema se apoya en una arquitectura cliente-servidor: un backend en Node.js/Express que expone una API REST, consumida por un frontend web con tres paneles diferenciados por rol de usuario, y una capa adicional de eventos en Google Cloud (Pub/Sub y Firebase Cloud Messaging).

A lo largo del documento se exponen: una introducción histórica a Oracle como compañía y como motor de base de datos; el proceso de instalación de Oracle Database Express Edition; los conceptos fundamentales de qué es, cómo funciona y para qué sirve un sistema gestor de bases de datos relacional; los diagramas de arquitectura general, despliegue, interacción, entidad-relación y secuencia del sistema; un diccionario de datos; fragmentos reales del código del servidor con su explicación; la matriz de roles y permisos; y finalmente, capturas de pantalla de la aplicación funcionando en sus distintos roles de usuario.

2. Historia de Oracle

Oracle Corporation es una de las compañías de software empresarial más influyentes en la historia de la computación, reconocida principalmente por haber popularizado el modelo de bases de datos relacionales en el mundo comercial. La compañía fue fundada en 1977 por Larry Ellison junto con Bob Miner y Ed Oates, originalmente bajo el nombre de Software Development Laboratories, antes de adoptar más adelante el nombre de su producto insignia, Oracle.

Uno de los primeros contratos de la naciente compañía fue el desarrollo de un sistema gestor de bases de datos para la CIA. La apuesta por construir ese sistema sobre el modelo relacional —teorizado por Edgar F. Codd en IBM— y consultable mediante el lenguaje SQL resultó decisiva: en 1981 IBM se convirtió en cliente de Oracle, lo que impulsó que las ventas de la startup se duplicaran cada año durante los siguientes siete años.

La compañía salió a bolsa en 1986 y, un año después, se convirtió en la mayor empresa de gestión de bases de datos del mundo. Con el paso de las décadas, Oracle Database evolucionó incorporando funcionalidades cada vez más sofisticadas: procesamiento de transacciones distribuidas, particionado de tablas, replicación, procesamiento en paralelo, soporte para datos semiestructurados (XML, JSON) y, más recientemente, integración con servicios de nube e inteligencia artificial.

2.1 Hitos relevantes en la evolución de Oracle

- **Fundación en 1977:** Larry Ellison, Bob Miner y Ed Oates fundan Software Development Laboratories, compañía que más adelante adoptaría el nombre de su producto principal, Oracle.
- **Adopción temprana por IBM (1981):** este acuerdo validó comercialmente el modelo relacional consultable mediante SQL y permitió que las ventas de la compañía se duplicaran año tras año durante varios años.
- **Salida a bolsa y liderazgo de mercado (1986-1987):** Oracle realizó su oferta pública inicial en 1986 y, un año después, se convirtió en la mayor empresa de gestión de bases de datos del mundo.
- **Ediciones para distintos contextos:** Oracle ofrece ediciones diferenciadas, desde Oracle Database Express Edition (XE), gratuita y orientada a aprendizaje y pequeños proyectos, hasta Oracle Database Enterprise Edition, pensada para cargas de producción críticas.
- **Evolución hacia la nube:** las versiones más recientes integran capacidades de nube (Oracle Cloud Infrastructure, Oracle Autonomous Database), automatizando tareas de administración como ajuste de rendimiento, parches y copias de seguridad — la misma Autonomous Database contemplada en el diagrama de despliegue de este proyecto (sección 7).

Relevancia para este proyecto

Para el Panel de Administración se utiliza Oracle Database Express Edition (XE) en el entorno de desarrollo local, una edición gratuita pensada para fines educativos que conserva el motor relacional completo de Oracle. El propio `backend.env` define `ORACLE_CONNECT_STRING=localhost:1521/XE` para esta fase, migrando a Oracle Autonomous Database (con wallet) al pasar a producción.

3. Instalación de Oracle Database

Para el desarrollo del proyecto se utiliza Oracle Database Express Edition (XE), pensada específicamente para fines de aprendizaje, desarrollo y proyectos de baja escala. El proceso de instalación, aunque varía ligeramente según el sistema operativo, sigue un flujo general de cinco etapas.

3.1 Paso 1 — Descarga del software

El proceso inicia accediendo al sitio web oficial de Oracle para descargar el instalador correspondiente, seleccionando la versión de Oracle Database XE adecuada para el sistema operativo (Windows o Linux) y aceptando los términos de licencia.

3.2 Paso 2 — Ejecución del instalador

Una vez descargado el archivo, se debe descomprimir y ejecutar el instalador (setup.exe en Windows) con privilegios de administrador, ya que el proceso requiere registrar servicios del sistema operativo y reservar puertos de red.

3.3 Paso 3 — Configuración de parámetros

El asistente solicita aceptar el acuerdo de licencia y seleccionar la ruta de instalación (por defecto, algo como C:\app\).

3.4 Paso 4 — Definición de contraseña

Uno de los pasos más sensibles es la creación de la contraseña para las cuentas administrativas SYS y SYSTEM. Es fundamental resguardar esta contraseña, ya que este mismo usuario SYSTEM es el que el proyecto usa como ORACLE_USER en desarrollo local (ver backend/.env.example).

3.5 Paso 5 — Finalización y verificación

El instalador completa la configuración del entorno e inicia los servicios necesarios. Es importante revisar que el puerto 1521 (el mismo definido en ORACLE_CONNECT_STRING=localhost:1521/XE del proyecto) esté correctamente asignado y disponible.

3.6 Verificación posterior a la instalación

Tras finalizar la instalación, se recomienda validar el funcionamiento del motor mediante una conexión de prueba con SQL Developer o SQL*Plus, y ejecutar después el script real del proyecto, oracle/schema.sql, que crea las tablas usuarios, stock y movimientos con datos de ejemplo.

```
-- Conexión de prueba mediante SQL*Plus
sqlplus system/contraseña@localhost:1521/XE

-- Cargar el esquema real del proyecto:
@oracle/schema.sql

-- Verificar que las tablas se crearon:
SELECT * FROM usuarios;
SELECT * FROM stock;
```

Paso	Acción principal	Consideración clave
1	Descarga del software	Verificar compatibilidad con el sistema operativo
2	Ejecución del instalador	Ejecutar con privilegios de administrador
3	Configuración de parámetros	Aceptar licencia y definir ruta de instalación

Paso	Acción principal	Consideración clave
4	Definición de contraseña	Anotar y resguardar las credenciales de SYS/SYSTEM
5	Finalización y verificación	Confirmar el puerto 1521 y ejecutar oracle/schema.sql

4. ¿Qué es, cómo funciona y para qué es Oracle Database?

4.1 ¿Qué es Oracle Database?

Oracle Database es un sistema de gestión de bases de datos relacionales (RDBMS), es decir, un software encargado de almacenar, organizar y administrar datos estructurados en forma de tablas compuestas por filas y columnas, manteniendo relaciones lógicas entre ellas mediante claves primarias y foráneas. Un RDBMS como Oracle garantiza las propiedades ACID: Atomicidad, Consistencia, Aislamiento y Durabilidad.

4.2 ¿Cómo funciona Oracle Database?

Internamente, Oracle Database opera mediante una instancia (procesos en memoria y de fondo) y archivos de base de datos en disco. Cuando el backend del proyecto envía una instrucción SQL —por ejemplo, el INSERT INTO movimientos que ejecuta movimientosService.js—, esta solicitud es recibida por el listener de red de Oracle, procesada por la instancia y persistida en los archivos de datos, registrándose además en archivos de redo log que permiten recuperar la base de datos ante una falla.

4.3 ¿Para qué es Oracle Database en este proyecto?

En el Panel de Administración, Oracle Database cumple el rol de fuente única de verdad (single source of truth) para toda la información crítica: los usuarios del sistema y sus roles, el inventario de stock disponible, y el historial de movimientos de entrada y salida. Cada vez que un vendedor registra una venta o un distribuidor registra una entrada de materia prima, esa operación se traduce en las instrucciones SQL de movimientosService.js, que actualizan de forma consistente y transaccional (INSERT + UPDATE + COMMIT) las tablas correspondientes.

Pregunta	Respuesta resumida
¿Qué es?	Un sistema gestor de bases de datos relacionales que almacena datos en tablas con integridad garantizada (ACID)
¿Cómo funciona?	Mediante una instancia en memoria, archivos de datos en disco y el lenguaje SQL para consultar y modificar la información
¿Para qué es?	Para ser la fuente única de verdad del inventario, usuarios y movimientos del Panel de Administración

5. Diagrama de Arquitectura General

El diagrama de arquitectura general describe, a nivel conceptual, cómo se organizan los componentes del sistema: el Cliente (frontend con sus 5 páginas HTML), el Servidor (backend Express con sus rutas, servicios y utilidades), la Base de Datos Oracle, y la capa de Eventos en Google Cloud.

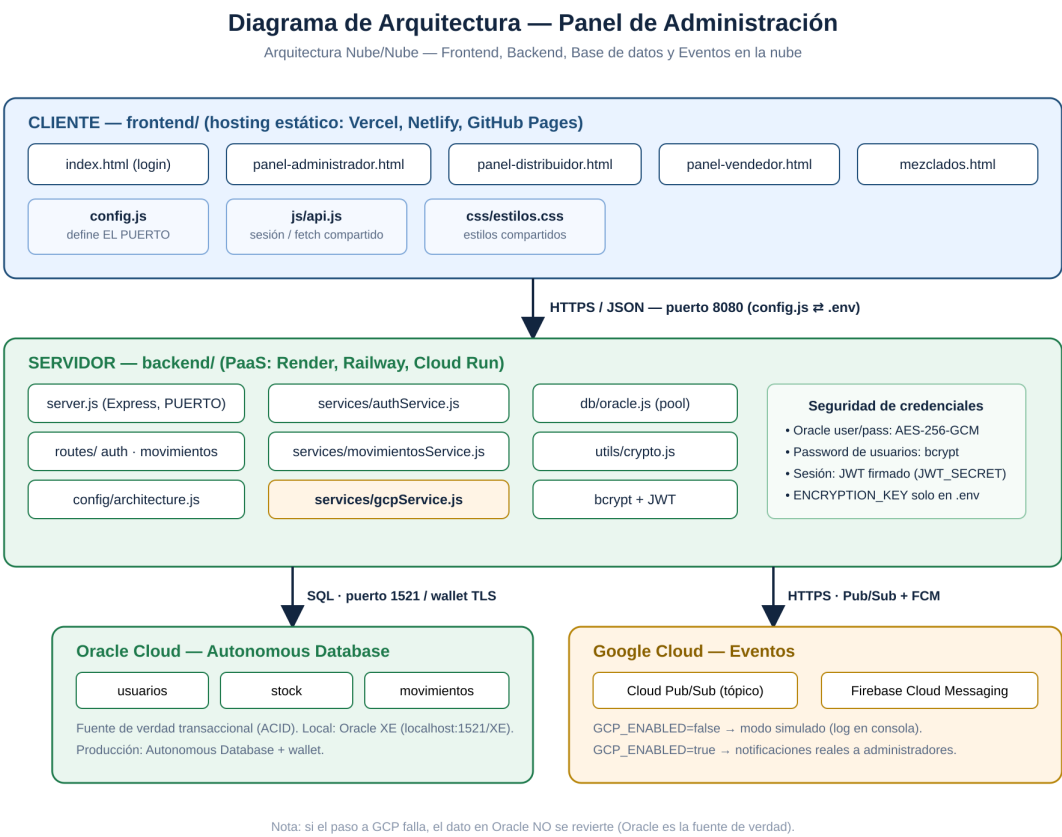


Figura 1. Diagrama de arquitectura general del sistema.

Capa	Responsabilidad principal	Archivos reales del proyecto
Frontend	Interfaz visual, login por rol, presentación de stock/movimientos	frontend/*.html, config.js, js/api.js
Backend	Reglas de negocio, validaciones, cifrado y autenticación	backend/src/routes, /services, /utils
Oracle Database	Almacenamiento persistente y transaccional (ACID)	oracle/schema.sql, backend/src/db/oracle.js
Google Cloud	Eventos y notificaciones desacopladas (best-effort)	backend/src/services/gcpService.js

Por qué separar estas capas

Separar el sistema en capas de Frontend, Backend, Oracle y GCP permite que cada una evolucione de forma independiente: la interfaz puede rediseñarse sin tocar la lógica de negocio, y la capa de eventos (GCP) puede fallar sin comprometer jamás el dato ya confirmado en Oracle, que es la fuente única de verdad.

6. Diagrama de Despliegue

Mientras que el diagrama de arquitectura describe los componentes lógicos, el diagrama de despliegue representa cómo esos componentes se distribuyen físicamente sobre dispositivos y servidores reales, siguiendo la notación UML de despliegue (dispositivos, entornos de ejecución y artefactos).

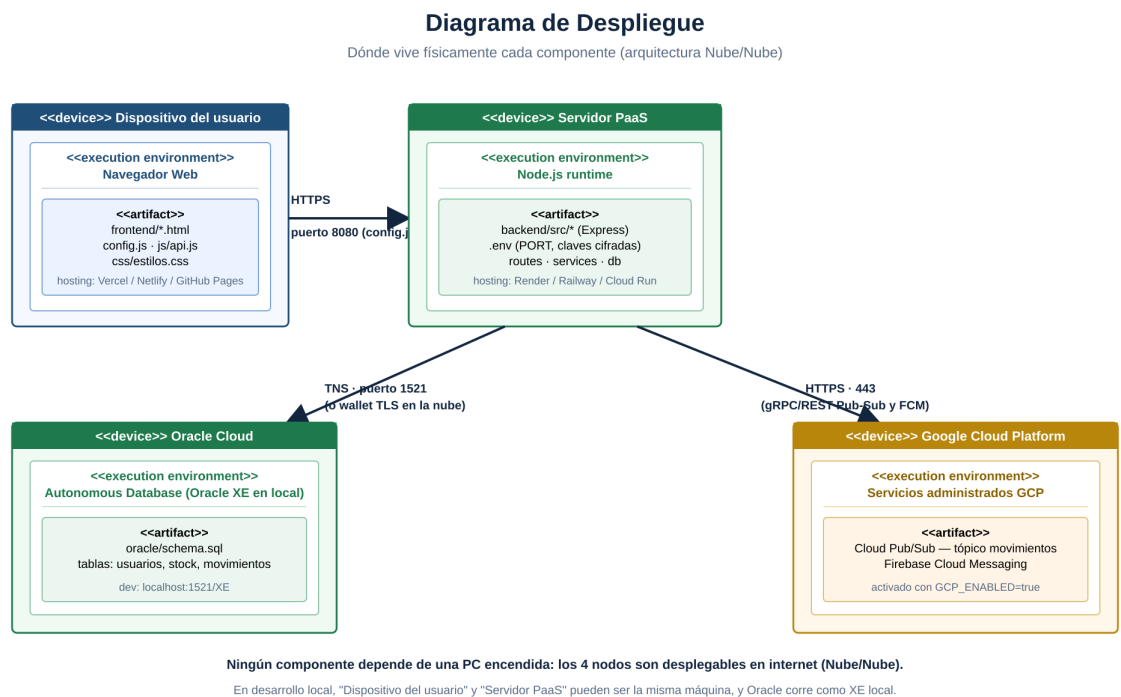


Figura 2. Diagrama de despliegue — nodos, entornos de ejecución y artefactos.

6.1 Nodos de la infraestructura

- **Dispositivo del usuario:** ejecuta el navegador web, que sirve los archivos estáticos de frontend/ (hospedados en Vercel, Netlify o similar).
- **Servidor PaaS:** ejecuta el runtime de Node.js con el código de backend/src, desplegado en un proveedor como Render, Railway o Google Cloud Run.
- **Oracle Cloud:** aloja la base de datos (Oracle XE en desarrollo, Autonomous Database en producción) con las tablas de oracle/schema.sql.
- **Google Cloud Platform:** aloja los servicios administrados de Pub/Sub y Firebase Cloud Messaging.

Origen	Destino	Protocolo / Puerto
Dispositivo del usuario	Servidor PaaS (Frontend → Backend)	HTTPS, puerto 8080 (config.js)
Servidor PaaS (Backend)	Oracle Cloud	TNS / puerto 1521 (o wallet TLS)
Servidor PaaS (Backend)	Google Cloud Platform	HTTPS / 443 (Pub/Sub + FCM)

7. Diagrama de Interacción

El diagrama de interacción presenta, en forma de diagrama de comunicación, cómo los tres actores del sistema —Administrador, Distribuidor y Vendedor— interactúan con los componentes técnicos. A diferencia del diagrama de secuencia de la sección 9, aquí el orden se indica con números sobre los enlaces en lugar de una línea de tiempo vertical.

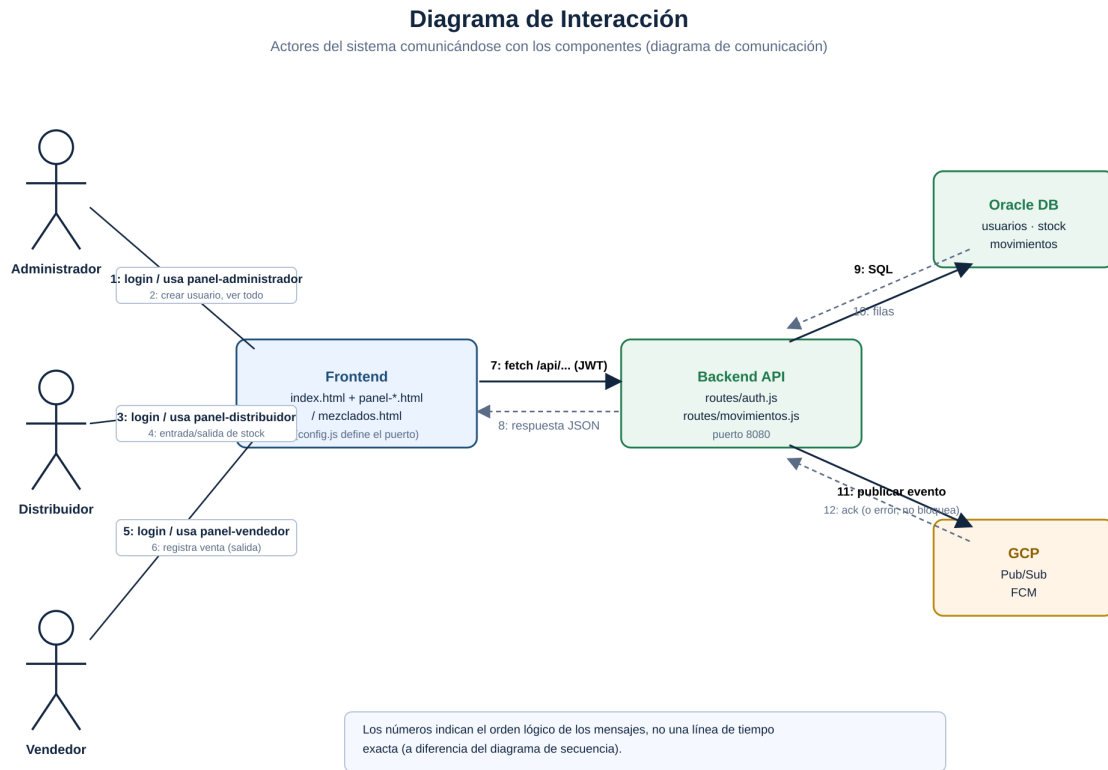


Figura 3. Diagrama de interacción entre actores y componentes del sistema.

7.1 Descripción paso a paso

- **1–2. Login:** cada actor envía sus credenciales desde index.html; el backend valida contra la tabla usuarios (bcrypt) y devuelve un JWT junto con el rol.
- **3–6. Uso del panel correspondiente:** el frontend usa ese rol para redirigir automáticamente a panel-administrador.html, panel-distribuidor.html o panel-vendedor.html.
- **7. Fetch autenticado:** cada panel llama a la API (con el JWT en el header Authorization) al puerto definido en config.js.
- **8. Respuesta:** el backend responde con JSON (stock, usuarios o movimientos, según el endpoint).
- **9–10. Backend ⇌ Oracle:** routes/movimientos.js delega en movimientosService.js, que ejecuta el SQL correspondiente contra Oracle.
- **11–12. Backend ⇌ GCP:** solo si Oracle confirmó, se intenta notificar a GCP; un fallo aquí nunca bloquea ni revierte el dato.

Importancia de este flujo

Este diagrama evidencia que la comunicación con GCP nunca bloquea la respuesta al usuario: es un paso adicional, posterior a que Oracle ya confirmó el cambio de forma transaccional.

8. Diagrama de Entidad-Relación

El diagrama de entidad-relación (DER) modela la estructura lógica de la base de datos del sistema, tal como está implementada en oracle/schema.sql, mostrando las tres entidades principales, sus atributos y las relaciones de cardinalidad entre ellas.

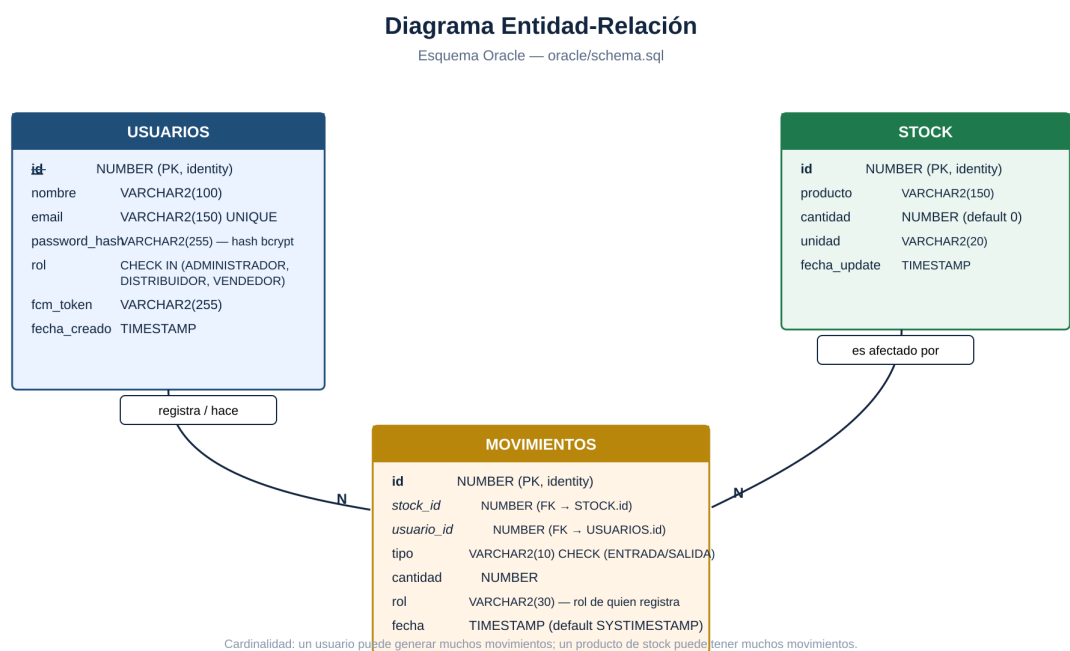


Figura 4. Diagrama entidad-relación del esquema Oracle.

8.1 Entidades principales

- **USUARIOS:** almacena los datos de acceso de cada persona que utiliza el sistema: id (PK), nombre, email, password_hash (bcrypt), rol y fcm_token. El atributo rol determina si el usuario actúa como ADMINISTRADOR, DISTRIBUIDOR o VENDEDOR.
- **STOCK:** representa el inventario de artículos: id (PK), producto, cantidad y unidad. El campo cantidad se actualiza en cada movimiento.
- **MOVIMIENTOS:** entidad central que registra cada entrada o salida de inventario: id (PK), stock_id (FK), usuario_id (FK), tipo (ENTRADA/SALIDA), cantidad, rol y fecha.

Relación	Cardinalidad	Significado
USUARIOS ↔ MOVIMIENTOS	1 a N	Un usuario puede registrar muchos movimientos
STOCK ↔ MOVIMIENTOS	1 a N	Un producto puede tener muchos movimientos asociados

Sobre las claves primarias y foráneas

PK identifica la clave primaria de cada tabla; FK identifica una clave foránea que referencia la clave primaria de otra tabla. En Oracle, estas relaciones se implementan mediante restricciones FOREIGN KEY (ver fk_mov_stock y fk_mov_usuario en oracle/schema.sql), garantizando que no pueda registrarse un movimiento que referencie un producto o usuario inexistente.

9. Diagrama de Secuencia

Este diagrama detalla, en orden cronológico estricto, el flujo completo de registrar un movimiento de stock (POST /api/movimientos), desde que el panel envía el formulario hasta que Oracle confirma la transacción y se notifica —opcionalmente— a Google Cloud. Incluye también el camino alternativo cuando el stock es insuficiente para una salida.

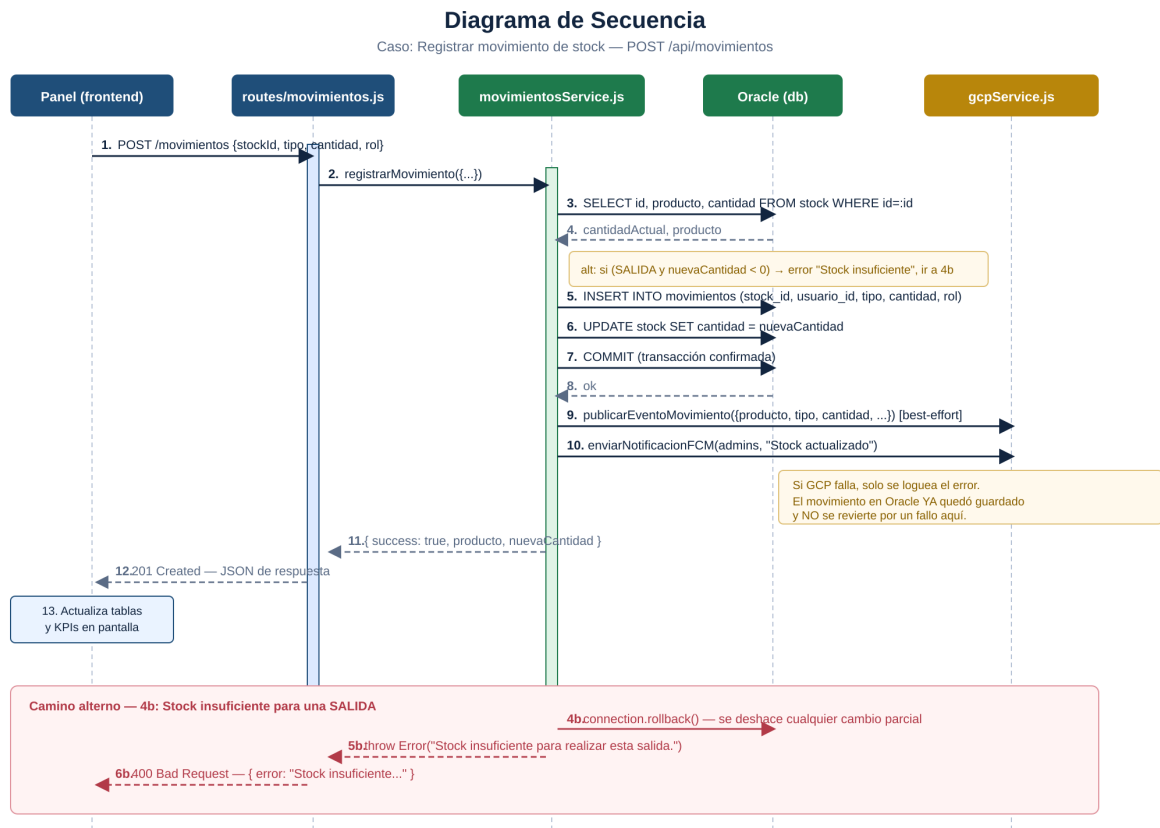


Figura 5. Diagrama de secuencia — flujo de registrar movimiento.

9.1 Descripción paso a paso

- **Pasos 1-2:** el panel (por ejemplo panel-vendedor.html) envía el formulario a routes/movimientos.js, que delega la lógica en movimientosService.js.
- **Pasos 3-4:** se valida que el producto exista y se obtiene la cantidad actual en Oracle mediante un SELECT.
- **Pasos 5-7:** se inserta el movimiento (INSERT), se actualiza el stock (UPDATE) y se confirma la transacción con COMMIT — todo dentro de la misma transacción para garantizar consistencia (ACID).
- **Pasos 9-10:** solo si Oracle confirmó con éxito, se intenta publicar el evento en Pub/Sub y enviar una notificación FCM a los administradores; un fallo aquí solo se registra en consola (gcpService.js).
- **Pasos 11-13:** la respuesta viaja de regreso hasta el panel, que actualiza las tablas y los indicadores en pantalla.
- **Camino alternativo (4b-6b):** si se intenta una SALIDA mayor al stock disponible, movimientosService.js hace rollback de la transacción y responde con un error 400, sin llegar a notificar a GCP.

Relación con el diagrama de interacción

Este diagrama de secuencia complementa al diagrama de interacción de la sección 7: mientras aquel muestra a los tres actores en conjunto, este se enfoca en el detalle técnico paso a paso de una sola operación crítica — registrar un movimiento — incluyendo su manejo transaccional real (rollback ante error).

10. Diccionario de Datos

El diccionario de datos documenta cada uno de los atributos que componen las tres tablas reales del sistema, definidas en oracle/schema.sql.

10.1 Tabla USUARIOS

Campo	Tipo de dato (Oracle)	Descripción
id	NUMBER (PK, identity)	Identificador único autogenerado del usuario
nombre	VARCHAR2(100)	Nombre completo del usuario
email	VARCHAR2(150) UNIQUE	Correo electrónico, usado como usuario de login
password_hash	VARCHAR2(255)	Hash bcrypt de la contraseña; nunca texto plano
rol	VARCHAR2(30) CHECK	ADMINISTRADOR, DISTRIBUIDOR o VENDEDOR
fcm_token	VARCHAR2(255)	Token del dispositivo para notificaciones push (FCM)
fecha_creado	TIMESTAMP	Fecha de alta del usuario (default SYSTIMESTAMP)

10.2 Tabla STOCK

Campo	Tipo de dato (Oracle)	Descripción
id	NUMBER (PK, identity)	Identificador único autogenerado del producto
producto	VARCHAR2(150)	Nombre comercial del producto
cantidad	NUMBER (default 0)	Cantidad disponible en inventario
unidad	VARCHAR2(20)	Unidad de medida (por defecto 'unidad')
fecha_update	TIMESTAMP	Última actualización de la cantidad

10.3 Tabla MOVIMIENTOS

Campo	Tipo de dato (Oracle)	Descripción
id	NUMBER (PK, identity)	Identificador único autogenerado del movimiento
stock_id	NUMBER (FK → stock.id)	Producto afectado por el movimiento
usuario_id	NUMBER (FK → usuarios.id)	Usuario que registró el movimiento
tipo	VARCHAR2(10) CHECK	ENTRADA o SALIDA
cantidad	NUMBER	Cantidad afectada por el movimiento
rol	VARCHAR2(30)	Rol de quien registró (auditoría adicional)
fecha	TIMESTAMP	Fecha y hora del movimiento (default SYSTIMESTAMP)

Convenciones utilizadas

PK identifica la clave primaria de cada tabla; FK identifica una clave foránea que referencia la clave primaria de otra tabla. Los tipos de datos corresponden a la nomenclatura propia de Oracle Database (VARCHAR2 en lugar de VARCHAR, y NUMBER como tipo numérico genérico para enteros y decimales).

11. Código del Sistema

A diferencia de una versión simulada con datos en memoria, el backend de este proyecto se conecta de verdad a Oracle Database mediante el driver oficial oracledb, con manejo transaccional real (commit/rollback) y cifrado de credenciales. A continuación se explican, con el código real del proyecto, los módulos más importantes.

11.1 server.js — Punto de entrada y EL PUERTO

El servidor Express centraliza la configuración del puerto (variable PORT, con 8080 como valor por defecto) y monta las rutas de la API bajo el prefijo /api.

```
const PORT = process.env.PORT || 8080;

app.use("/api", movimientosRoutes);
app.use("/api", authRoutes);

async function start() {
  await initPool();      // conecta a Oracle antes de aceptar peticiones
  initGcpClients();      // inicializa GCP (real o simulado)
  app.listen(PORT, () => { /* ... */ });
}
```

11.2 utils/crypto.js — Cifrado de credenciales de Oracle

Las credenciales de conexión a Oracle (usuario y contraseña) se cifran con AES-256-GCM antes de guardarse en el archivo .env, usando una clave maestra (ENCRYPTION_KEY) que nunca cifra datos de negocio, solo estas credenciales.

```
const ALGORITHM = "aes-256-gcm";

function encrypt(plainText) {
  const key = getKey();
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);
  const encrypted = Buffer.concat([cipher.update(String(plainText), "utf8"),
  cipher.final()]);
  const authTag = cipher.getAuthTag();
  return [iv.toString("hex"), authTag.toString("hex"), encrypted.toString("hex")].join(":");
}
```

El uso de GCM (en vez de CBC) agrega un authTag: si alguien altera el valor cifrado guardado en .env, decrypt() falla explícitamente en vez de devolver datos corruptos silenciosamente.

11.3 services/authService.js — Registro y login

Las contraseñas de los usuarios de la aplicación (a diferencia de las credenciales de Oracle) se guardan con hash bcrypt, que no es reversible: ni el propio backend puede recuperar la contraseña original.

```
const passwordHash = await bcrypt.hash(password, SALT_ROUNDS); // registro

// login:
const passwordValida = await bcrypt.compare(password, usuario.PASSWORD_HASH);
if (!passwordValida) throw new Error("Credenciales invalidas.");

const token = jwt.sign(
  { id: usuario.ID, email: usuario.EMAIL, rol: usuario.ROL },
  getJwtSecret(),
  { expiresIn: "8h" }
```

```
);
```

11.4 services/movimientosService.js — Lógica transaccional

Esta es la función central del sistema: registra un movimiento y actualiza el stock como una única transacción atómica, revirtiendo todo (rollback) si el stock resulta insuficiente.

```
const nuevaCantidad = tipo === "ENTRADA"
  ? cantidadActual + cantidad
  : cantidadActual - cantidad;

if (nuevaCantidad < 0) {
  throw new Error("Stock insuficiente para realizar esta salida.");
}

await connection.execute(`INSERT INTO movimientos (...) VALUES (...)`, {...});
await connection.execute(`UPDATE stock SET cantidad = :nuevaCantidad ...`, {...});
await connection.commit(); // confirma ambas operaciones como una unidad

// ... si algo falla antes del commit:
// await connection.rollback();
```

11.5 services/gcpService.js — Eventos desacoplados

La notificación a Google Cloud se intenta después de que Oracle ya confirmó el cambio, y su fallo nunca revierte el dato ya guardado (principio de "best-effort").

```
try {
  await publicarEventoMovimiento(eventoPayload);
  // ... enviarNotificacionFCM(...) a los administradores
} catch (gcpError) {
  // Importante: un fallo en la notificacion NUNCA revierte Oracle.
  console.error("[GCP] Error al notificar:", gcpError.message);
}
```

12. Roles y Permisos

El sistema segmenta el acceso en tres roles, cada uno con su propio panel en el frontend y sus propias restricciones de negocio.

Rol	Puede hacer	No puede hacer
Administrador	Ver stock, usuarios y movimientos completos; crear usuarios; registrar cualquier movimiento	—
Distribuidor	Registrar entradas y salidas de stock; ver inventario e historial	Crear usuarios; ver la lista completa de usuarios
Vendedor	Registrar salidas (ventas); ver stock disponible y sus propias ventas	Crear usuarios; registrar entradas; ver usuarios ni movimientos de otros

Principio de seguridad aplicado

Esta segmentación de permisos por rol —reforzada en el frontend por `protegerPagina(rolesPermitidos)` en `js/api.js`— es una aplicación directa del principio de privilegio mínimo: cada actor del sistema solo puede ejecutar las operaciones estrictamente necesarias para cumplir su función.

13. Capturas de Pantalla del Sistema

A continuación se presentan capturas reales del frontend del proyecto (frontend/*.html), renderizadas con datos de muestra equivalentes a los que carga oracle/schema.sql, para ilustrar el funcionamiento de cada rol.

13.1 Inicio de sesión (index.html)

Punto de entrada único del sistema: valida credenciales contra POST /api/auth/login y redirige automáticamente al panel correspondiente según el rol devuelto.

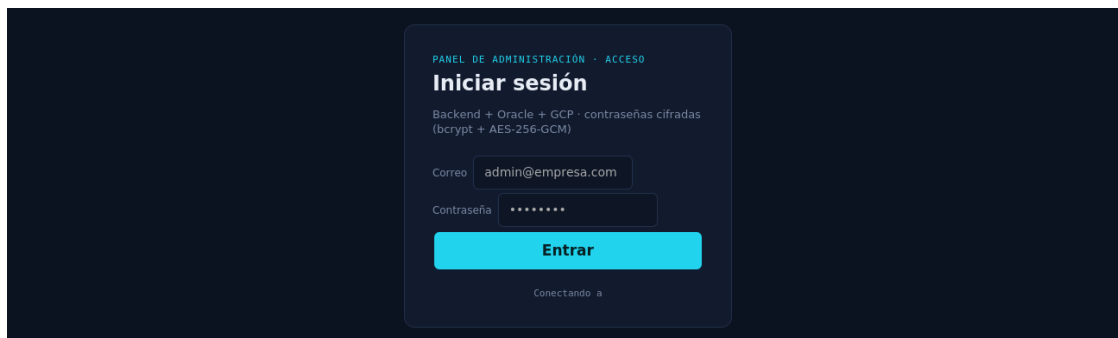


Figura 6. Pantalla de login — indicador de conexión al backend en el pie.

13.2 Panel del Administrador

Despliega tres indicadores (productos, usuarios y movimientos), el inventario completo, un formulario para crear usuarios (contraseña cifrada con bcrypt) y el historial completo de movimientos.

Panel Administrador

ADMINISTRADOR

●conectando...

Cerrar sesión

PRODUCTOS EN STOCK

USUARIOS REGISTRADOS

MOVIMIENTOS TOTALES

INVENTARIO (STOCK)

ID	PRODUCTO	CANTIDAD	UNIDAD
----	----------	----------	--------

CREAR NUEVO USUARIO

Nombre

Correo

Contraseña (se guarda cifrada/hasheada)

Rol

ADMINISTRADOR

Crear usuario

USUARIOS

ID	NOMBRE	CORREO	ROL
----	--------	--------	-----

REGISTRAR MOVIMIENTO

Producto (stock)

Tipo

ENTRADA

Cantidad

Registrar

HISTORIAL DE MOVIMIENTOS

ID	PRODUCTO	TIPO	CANTIDAD	ROL	FECHA
----	----------	------	----------	-----	-------

Figura 7. Panel del Administrador con datos de muestra.

13.3 Panel del Distribuidor

Enfocado en la operación de almacén: registrar entradas/salidas de stock y consultar el inventario e historial, sin acceso a la gestión de usuarios.

Panel Distribuidor

DISTRIBUIDOR

●conectando...

Cerrar sesión

REGISTRAR MOVIMIENTO DE STOCK

Producto

Tipo

Cantidad

ENTRADA

Registrar

INVENTARIO ACTUAL

ID	PRODUCTO	CANTIDAD	UNIDAD
----	----------	----------	--------

HISTORIAL DE MOVIMIENTOS

ID	PRODUCTO	TIPO	CANTIDAD	FECHA
----	----------	------	----------	-------

Figura 8. Panel del Distribuidor.

13.4 Panel del Vendedor (punto de venta)

Interfaz orientada a ventas: selecciona un producto, registra la cantidad vendida (salida de stock) y consulta sus propias ventas recientes.

Panel Vendedor

●conectando...

Cerrar sesión

REGISTRAR VENTA (SALIDA DE STOCK)

Producto

Cantidad vendida

Registrar venta

STOCK DISPONIBLE

PRODUCTO	DISPONIBLE	UNIDAD
----------	------------	--------

MIS ÚLTIMAS VENTAS REGISTRADAS

PRODUCTO	CANTIDAD	FECHA
----------	----------	-------

Figura 9. Panel del Vendedor.

13.5 Panel combinado (mezclados.html)

Vista unificada de stock, usuarios y movimientos, accesible para cualquier rol logueado; útil como panel de auditoría general del sistema.

Panel Combinado

●conectando...

Cerrar sesión

PRODUCTOS

USUARIOS

MOVIMIENTOS

REGISTRAR MOVIMIENTO

Producto

Tipo

ENTRADA

Cantidad

Guardar

STOCK

PRODUCTO	CANT.	UNIDAD
----------	-------	--------

USUARIOS

NOMBRE	CORREO	ROL
--------	--------	-----

HISTORIAL DE MOVIMIENTOS

PRODUCTO	TIPO	CANTIDAD	ROL	FECHA
----------	------	----------	-----	-------

Figura 10. Panel combinado mezclados.html.

14. Conclusiones

El Panel de Administración desarrollado ilustra, de manera integral, cómo Oracle Database puede emplearse como motor central de un sistema de información de pequeña y mediana escala. A lo largo de este documento se repasó la historia de Oracle Corporation, el proceso de instalación de Oracle Database Express Edition, y los conceptos fundamentales de qué es, cómo funciona y para qué sirve un RDBMS relacional.

Desde el punto de vista de diseño, los cinco diagramas presentados —arquitectura general, despliegue, interacción, entidad-relación y secuencia— mostraron cómo un mismo sistema puede analizarse desde perspectivas complementarias: los componentes lógicos involucrados, su distribución física sobre dispositivos y servidores reales, la forma en que los actores se comunican con esos componentes, el modelo de datos concreto (USUARIOS, STOCK, MOVIMIENTOS), y el detalle paso a paso del proceso crítico de registrar un movimiento, incluyendo su manejo transaccional real.

En el plano de la implementación, se presentó el código real del servidor Node.js con conexión genuina a Oracle Database mediante el driver oracledb, incluyendo el uso de un pool de conexiones, cifrado AES-256-GCM de credenciales, hash bcrypt de contraseñas de usuario, JWT para sesiones, y manejo transaccional con commit y rollback para garantizar que el registro de movimientos y la actualización de stock ocurran siempre como una unidad atómica.

Finalmente, las capturas de pantalla del sistema en funcionamiento permitieron verificar que la segmentación de roles —Administrador, Distribuidor y Vendedor— efectivamente se traduce en vistas e interfaces diferenciadas, aplicando de forma consistente el principio de privilegio mínimo. En conjunto, este proyecto demuestra que es posible construir un sistema completo y coherente que combine buenas prácticas de modelado de datos, diseño de arquitectura por capas y manejo transaccional confiable sobre Oracle Database.